

TorchKart: Training an AI Agent to Play Mario Kart 64 Using Deep Reinforcement Learning

Jacob Terkuc

Introduction

Background

Reinforcement learning for video games is a well-studied domain. There is the Atari/DQN (Mnih et al., 2013), StarCraft/AlphaStar (Vinyals et al., 2019), and Dota/OpenAI Five (OpenAI et al., 2019), but racing games are relatively unexplored, especially retro games with no native control API's.

Mario Kart 64 presents a challenging environment for an agent:

- Continuous racing over multiple laps
- No "rails" to keep a player on the track
- Drifting mechanics
- Item Usage
- No pre-built API/training interface

Last semester, I worked on the first version of TorchKart, where I developed the base reinforcement learning pipeline connecting the Bizhawk emulators to a Python server over TCP sockets. This implementation had a primitive PPO scheme with a multi-layer perceptron and frame stacking as a way for an agent to have temporal awareness of its environment.

Due to the limitations imposed by only having 8-frame stacking (only about 0.13 seconds at 60fps), the agent was not able to "see" far enough to plan multi-step actions like drifts (which take 3+ seconds to execute: initiate -> turn in -> boost), and would waste the three "speed mushrooms" given to it at the beginning of the race. The agent also had no observation of drift state, mushroom boost status, or the number of mushrooms left, and conversely had no reward signals for these behaviors.

Objectives

My objectives I aimed to achieve for this iteration were as follows:

- Extend the observation space with drift state, mushroom boost, mushroom count, and derived features (drift duration, angular velocity) so the agent can “see” what it needs to learn advanced mechanics in a limited feature space
- Design targeted reward shaping for drift boosts and mushroom timing to incentivize strategic behaviour
- Replace the MLP + frame stacking with a causal Transformer architecture that gives the agent ~34 second temporal context window (512 step sequence length * 4x frame skip), enabling long-horizon planning for drift sequences
- Implement behavioural cloning pretraining on human demos to make better use of limited compute
- Redesign the action encoding from single-button to multi-discrete vectors [`throttle(3)`, `steering(5)`, `drift(2)`, `item(2)`] to enable simultaneous button combinations.
- Validate improvements by comparing against a human baseline and the previous semester’s baseline, targeting a measurable lap time reduction

Prior Work

Schuman et al. (2017) - Proximal Policy Optimization

Introduced PPO, a policy gradient method for reinforcement learning that clips objective ratios to prevent destructively large policy updates, balancing sample efficiency with training stability.

PPO is the core training algorithm used in TorchKart. It was chosen over DQN for its stability, compatibility with parallel environments, and for its performance in complex action spaces.

Schuman et al. (2016) - GAE

Introduced Generalized Advantage Estimation, allowing for stable learning in high-dimensional environments.

GAE is used for advantage estimation, providing a low-variance, low-bias signal allowing PPO to be stable across 20 parallel environments.

Mnih et al. (2013) - DQN / Atari

Introduced a deep learning model that can learn control policies directly from high-dimensional input using RL, tested on Atari 2600 games.

This paper pioneered RL for games, but used pixel input and a flat action space. They used the [Arcade Learning Environment](#), a precursor to [Gymnasium](#), which TorchKart uses. TorchKart differs by using targeted features from direct memory access, and uses multi-discrete actions.

Vaswani et al. (2017) - Attention is All You Need

Foundational paper for the Transformer, based solely on attention mechanisms. Shows superior performance with less training time when compared to recurrence or convolution based neural networks.

Architectural basis for TorchKart's causal Transformer. Uses the self-attention mechanism with causal masking for temporal reasoning in a racing task. 3 layers, 4 heads, 128-dim embeddings instead of the original NLP-scale configuration.

Vinyals et al. (2019) - AlphaStar

Reinforcement learning at scale, proves that transformers + IL can work for complex games.

AlphaStar combines Transformers, imitation learning pretraining and reinforcement learning fine tuning for a complex game, the same three-stage pipeline that TorchKart uses.

Pomerleau (1989) - ALVINN

The Autonomous Land Vehicle In a Neural Network (ALVINN) project is an early demonstration of learning driving behaviours from human demonstration.

TorchKart's imitation learning component learns driving behaviour from human demonstration, before reinforcement learning fine-tuning.

Kevinhughes27 (2017) - TensorKart

Self driving Mario Kart 64 machine learning project. Uses supervised learning on screen pixels to mimic human play.

This project was an inspiration for the original TorchKart implementation, but the approach is fundamentally different. TorchKart uses memory-reading feature extraction, multi-discrete actions, and a transformer, training an agent that discovers its own strategies rather than emulating human inputs.

Methods

PPO:

TorchKart uses a PPO mechanism rather than a DQN. The reasoning for this decision comes down to two main benefits: the gradient clipping mechanism, and its ability to handle discrete actions and parallel environments. . Both of these improvements lead to faster, smaller, more consistent policy updates, and were proven in the previous implementation of TorchKart. After ~200M timesteps, the DQN agent was unable to complete one lap around the track, where the PPO agent was able to complete three laps in ~10M steps.

Multi-Discrete Action Space:

The old system sent one series of button presses at a time, and could pick between a series of pre-determined button sequences. The new system allows the model to press any combination of buttons it feels is appropriate by using separate output heads for each group of controls:

- Throttle: Nothing, Acceleration(A), Brake(B)
- Steering: Hard left, slight left, center, slight right, hard right
- Drift: No drift, drift(R)
- Item: No item, use item(Z)

This also introduces significant statistical efficiency, whereas before when using a flat action space the same number of control combinations would make up 60 combinations (one output over 60 classes), the agent now has 12 total outputs over 4 heads, meaning that each head independently learns its action distribution.

Causal Transformer:

The old MLP had ~0.12 seconds of context, not enough to plan a drift sequence (3+ seconds). The transformer gives 512-frame context (~34 seconds with 4x frame skip) allowing for the agent to plan ahead for advanced maneuvers, and the causal masking ensures no future information leakage.

New Observations:

Added memory-backed observations `drift_state`, `mushroom_count`, and `mushroom_boost` that tracked the kart's active drift state, mushroom count, and boost state, and derived `drift_duration` and `angular_velocity`, allowing for the agent to “see” its movement in 3D space, allowing it to execute advanced maneuvers like drifting.

`Drift_duration` and `angular_velocity` are not readily available in the game's RAM, but are calculated using frame-to-frame deltas computed during feature normalization in the `_process_state` function.

Reward Shaping:

Adding explicit signals for drift boosting and mushroom use gives the agent an incentive to learn these new behaviours. The old system only had speed and progress rewards, so it was not incentivised to use advanced techniques to reduce lap times.

Imitation Learning/Behavioural Cloning:

Pre-training on human demos allows the agent to bypass the ~30M steps it would normally take before it makes one lap around the track. Behavioural cloning gives it basic racing skills from step 0, improving training efficiency and convergence time.

GAE:

$\gamma=0.99$, $\lambda=0.95$ for advantage estimation.

Environment & Dataset

Environment: Mario Kart 64 running in Bizhawk Emulator (v2.11, x86, Windows)

Track: Luigi's Circuit, complex map geometry with multiple points to get stuck on, many varying curves to trial drifting mechanics.

State Space: 14 normalized features from raw memory:

Velocities (x,y,z), Speed, Heading (sin/cos), Distance from center of track, Track Progress, Lap #, Mushroom Count, Wall Hit State, Drift state, Drift Duration, Mushroom Boost state, Angular Velocity

Frame Skip: 4x (agent acts every 4 frames), speeds up training significantly with no visible loss in performance.

Observation Stacking: 8 frames (MLP), 512-frame context window (Transformer)

Protocol: 50 bytes per state sent over TCP

Human Demo Dataset: 7 Runs, each with three laps of gameplay for behavioral cloning. Each demo records inputs and the proceeding reward output used for pre-training. Run average of ~115 seconds.

Validation and Experiments

The primary validation will be comparing the performance of the driving model against two runs:

Run 1 (Baseline): MLP + Frame Stacking + old observations + old reward shaping + single-button actions

Run 2 (Improved): Transformer + IL pretraining + new observations + new reward shaping + multi-discrete action vectors

The primary validation metric used to show model improvement will be race completion time (in seconds).

Tensorboard was used to track performance metrics with a rolling mean window of 200 samples. The following metrics were collected:

Episode reward over time (learning curve), Episode length over time, Average speed, Final lap achieved, Wall hits per episode, Drift boosts per episode, Mushrooms used / wasted per episode, Policy entropy per action head (throttle, steering, drift, item), Training Health Metrics (Value loss, policy loss, explained variance), Training speed (steps per second)

To validate if the addition of imitation learning made a significant impact in convergence time, a comparison between the step count of the first run to complete three laps between the two runs will be made, as well as a comparison with other key validation metrics.

Results

The improved system was able to achieve a best race time **10 seconds faster** than the baseline.

Below are the results compared between the two runs:

Metric	MLP Baseline	Transformer + IL	Change
Best Race Completion Time (s)	117.7s	107.7s	-10s
Average Race Completion (At termination) (s)	119.8s	109.6s	-10.2s
Best Episode Reward	1725 pts	1934 pts	+209 pts
Average Speed (At termination)	61.66	67.8	+6.14
Average Wall Hits/Episode (At Termination)	5.04	0.80	-4.24
Steps to First Race Completion	9,939,450	1,456,060	-8,483,390

Fig.1: Table of baseline vs improved model comparisons. Metrics labelled as “Average” are smoothed using a rolling mean window of 200 samples.

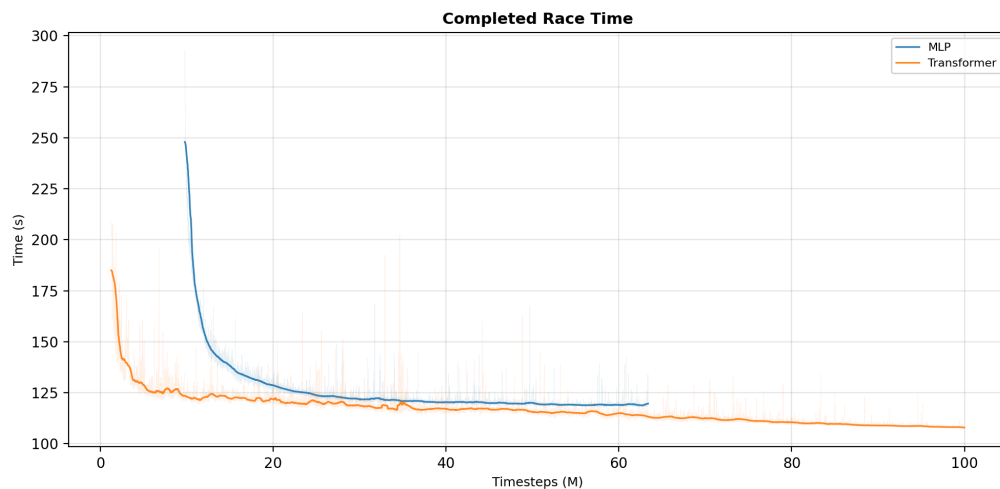


Fig. 2: Chart showing completed race time in seconds over timesteps in millions comparing the MLP and Transformer models. Solid line is using a rolling mean window of 200 samples.

As you can see in fig. 2, the improved model not only reaches a lower completed race time over its entire lifespan compared to the MLP agent, but it also completes its first lap 6.8x faster thanks to its improved architecture and the use of the imitation learning pre-training.

The old MLP agent could drive forward, follow the track and complete a race, but took wide turns, couldn't drift, used items randomly, and essentially learned to hold the accelerator and turn wildly. Steering could be described as 'jittery', and the agent would often let go of the accelerator to press a different random button combination. It was unable to find optimal racing lines, as it was unable to remember where it was ~ 0.1 seconds prior.

When comparing its driving performance to the Transformer agent, improvements are easily observable, such as its ability to start, initiate, turn in, and release a drift (multi-step planning), waiting for the right time to use a mushroom. The agent generally drives much more naturally, with long, controlled turns and applying the accelerator/steering to correct mid-drift to stay on the track. The transformer agent is able to find and optimize its racing lines thanks to a context length that can fit ~ 1 full lap worth of driving. At initialization, the Transformer agent has a much better sense of basic driving skills thanks to its pre-training when compared to the MLP agent. This is also shown in fig.2, illustrated by the significant gap in first race completion.

Discussion

The Transformer architecture was first demonstrated by Vaswani et al. for natural language processing tasks, but generally works for any ordered data. This property makes it an excellent choice for reinforcement learning applications such as Mario Kart, but requires the addition of a causal mask to prevent the model from peaking into the future. The model can learn which past frames are most relevant in the current state rather than treating them all the same, and is more performant on large environments and action sequences than LSTM's, which become saturated and lose performance.

This combined with other changes such as a multi-discrete action space, allowing for wider margins in the action distribution for each action; The imitation learning pre-training allowing the agent to learn basic driving mechanics much faster; improved reward shaping incentivizing drifting and mushroom use mechanics; All show an overall lap time improvement of ~10 seconds.

A few tests had to be cut from this assignment due to timing and compute constraints. Each run took about a week to train fully, and even though I had trained it on a GPU with 12GB RAM, anything over a sequence size of 512 would cause OOM errors, limiting overall training scale. Due to this, the model's performance on 1024-step sequences remains untested. The model's generalized performance on other tracks has not been tested. The model's performance in dynamic environments with other agents is currently untested.

In the future, it would be interesting to test the agent's performance with 1-3 other racers on a track, and see if they are able to learn new behaviours to beat out the competition. Parsing the emulator's RAM values for track segment data, distance between the front of the kart and a wall, and other features could improve performance going forward, but I am quite happy with the improvements that have been made. This work also demonstrates the improvements of using a Transformer architecture over a multi-layer perceptron in tasks that require long term temporal awareness in reinforcement learning tasks.

References

- kevinhughes27. (n.d.). Kevinhughes27/TensorKart: Self-driving Mariokart with TensorFlow. GitHub. <https://github.com/kevinhughes27/TensorKart>
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013, December 19). Playing Atari with deep reinforcement learning. arXiv.org. <https://arxiv.org/abs/1312.5602>
- OpenAI, Berner, C., Brockman, G., Chan, B., Cheung, V., Dębiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., Józefowicz, R., Gray, S., Olsson, C., Pachocki, J., Petrov, M., Pinto, H. P. d. O., Raiman, J., Salimans, T., ... Zhang, S. (2019, December 13). Dota 2 with large scale deep reinforcement learning. arXiv.org. <https://arxiv.org/abs/1912.06680>
- Pomerleau, D. (1991). Efficient Training of Artificial Neural Networks for Autonomous Navigation. https://www.ri.cmu.edu/pub_files/pub3/pomerleau_dean_1991_1/pomerleau_dean_1991_1.pdf
- Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2018, October 20). High-dimensional continuous control using generalized advantage estimation. arXiv.org. <https://arxiv.org/abs/1506.02438>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017, August 28). Proximal policy optimization algorithms. arXiv.org. <https://arxiv.org/abs/1707.06347>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023, August 2). Attention is all you need. arXiv.org. <https://arxiv.org/abs/1706.03762>
- Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., Vezhnevets, A. S., ... Silver, D. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782), 350–354. <https://doi.org/10.1038/s41586-019-1724-z>